ANA675 Assignments

Project Repo

Christopher Doupis

December 6, 2026

AI USE DISCLAIMER & ACADEMIC INTEGRITY COMPLIANCE

In compliance with the **National University Student Guidelines for Generative AI**, this project discloses the following tool usage:

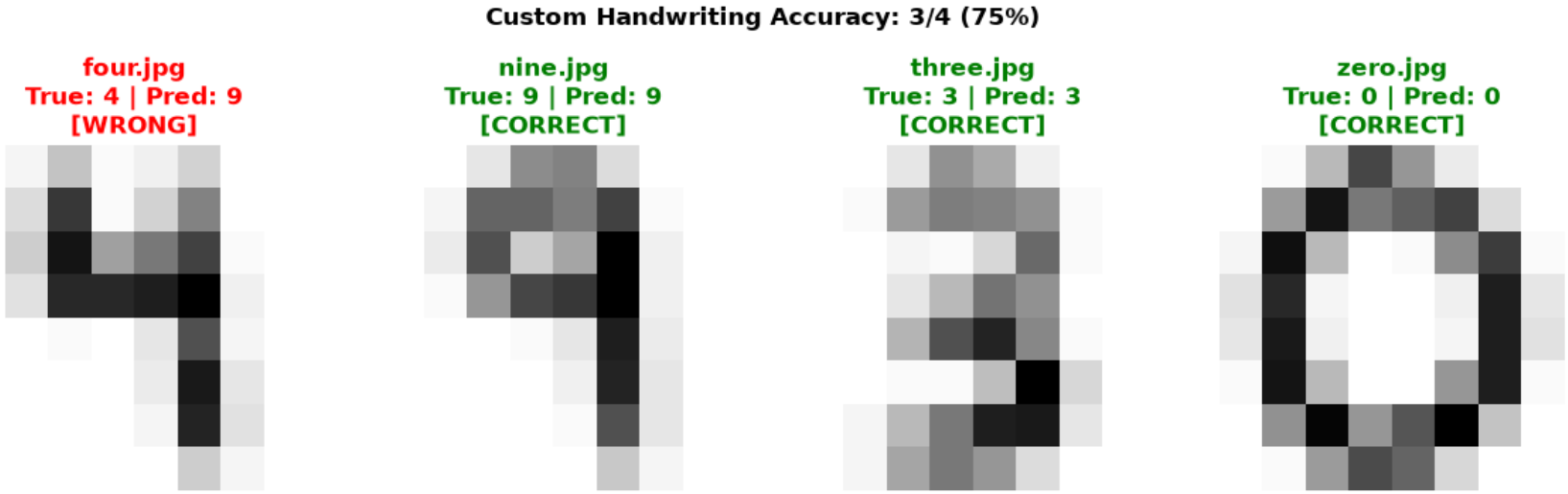
- **Conceptual Framing & Brainstorming (AI-Informed): Google Gemini** was utilized as a supplemental learning resource to clarify core mathematical and analytical concepts across the assignment questions.
- **Technical Execution & Boilerplate Generation (AI-Assisted): Google Gemini** was used to generate initial code structures and programming boilerplates. These templates were subsequently modified, debugged, and tailored to meet the specific requirements of each question.
- **Human Oversight, Validation Loops & Refinement (Author-Owned):** The author maintained complete conceptual control, process oversight, and ownership of the entire homework scope. All AI-assisted templates, code logic, and conceptual outputs were systematically reviewed and validated by the author to ensure academic accuracy and integrity.
- **Disclosure Statement (AI-Generated): Google Gemini** was utilized to structure and audit this text format to fulfill all institutional reporting standards.

REFERENCES

Peer Reviewed and Scientific References

Generative AI & Software Tools

- **Google.** (2026). *Gemini* [Large language model]. https://google.com



Assignment 2: XOR Logic Gate with Keras MLP (Backpropagation)

Goal

- **Objective:** Use **Keras** (with the PyTorch CPU backend) to design, train, and evaluate a **Multi-Layer Perceptron (MLP)** neural network using the **Backpropagation algorithm** to function as a two-input **XOR gate**.
- **Logic Rule:** Output `1` only when the two binary inputs differ (`(0, 1) = 1` , `(1, 0) = 1`), and output `0` when the inputs are identical (`(0, 0) = 0` , `(1, 1) = 0`).
- **Deliverables:** Show the neural network architecture, hyperparameter tuning results, final loss/accuracy evaluation, and binary predictions across all four truth-table combinations.

1. Create & Inspect the XOR Truth Table Dataset

In [8]:
```
os.makedirs("data", exist_ok=True)
xor_path = "data/xor_dataset.csv"
xor_df = pd.DataFrame({
    "x1": [0, 0, 1, 1],
    "x2": [0, 1, 0, 1],
```

```
    "target_xor": [0, 1, 1, 0],
})
xor_df.to_csv(xor_path, index=False)
display(xor_df)
```

	x1	x2	target_xor
0	0	0	0
1	0	1	1
2	1	0	1
3	1	1	0

2. Design, Tune & Train the Keras MLP (Backpropagation)

In [9]:
```
pipeline.run_generalized_pipeline("xor_mlp")
```

```
--- Running [PRODUCTION] Validation For: XOR_MLP (run_7a3bb635) ---
Source: data/xor_dataset.csv
Success! Processed XOR_MLP | Final Accuracy: 1.0000
```

Out[9]: True

3. Evaluate the Model & Display XOR Predictions

In [13]:
```
logs_df = pd.read_csv(os.path.join(config["paths"]["log_dir"], config["paths"].get("log_file", "experiment_
latest_id = logs_df[logs_df["engine_type"] == "xor_mlp"].iloc[-1]["run_uuid"]
bundle = utils.load_bundle(config["paths"]["model_dir"], latest_id, "xor_mlp", config.get("test_mode", Fals

xor_model = bundle["model"]
xor_model.summary()

X_xor, y_xor = xor_df[["x1", "x2"]], xor_df["target_xor"]
loss, accuracy = xor_model.evaluate(X_xor, y_xor, verbose=0)
display(pd.DataFrame([{"Model": "XOR MLP", "Run UUID": latest_id, "Best Parameters": str(bundle["metadata"]

raw_probs = xor_model.predict(X_xor, verbose=0).flatten()
errors = y_xor - raw_probs
```

```
sse = round(float((errors ** 2).sum()), 6)

slide_df = pd.DataFrame({
    "x1": xor_df["x1"],
    "x2": xor_df["x2"],
    "Desired output (yd)": y_xor,
    "Actual output (y)": raw_probs.round(4),
    "Error (e)": errors.round(4),
    "Predicted class": (raw_probs >= 0.5).astype(int),
    "Sum of squared errors": [sse, "", "", ""],
})
display(slide_df)
```

Model: "sequential_8"

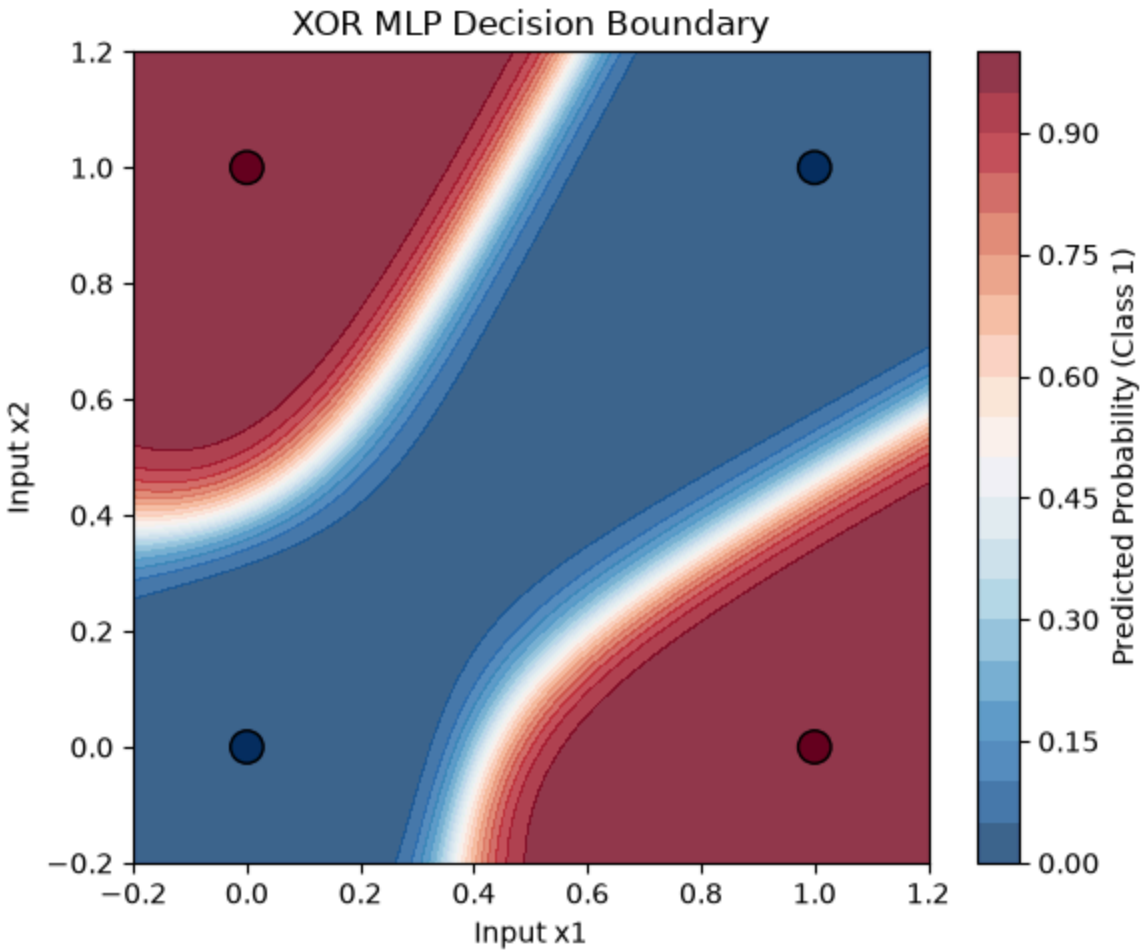
Layer (type)	Output Shape	Param #
dense_16 (Dense)	(None, 8)	24
dense_17 (Dense)	(None, 1)	9

Total params: 101 (404.00 B)
Trainable params: 33 (132.00 B)
Non-trainable params: 0 (0.00 B)
Optimizer params: 68 (272.00 B)

	Model	Run UUID	Best Parameters	Final Loss	Final Accuracy
0	XOR MLP	run_7a3bb635	{'hidden_activation': 'tanh', 'hidden_units': ...	0.0003	100.00%

	x1	x2	Desired output (yd)	Actual output (y)	Error (e)	Predicted class	Sum of squared errors
0	0	0	0	0.0001	-0.0001	0	0.0
1	0	1	1	0.9995	0.0005	1	
2	1	0	1	0.9997	0.0003	1	
3	1	1	0	0.0003	-0.0003	0	

In [11]:
```
visualization.plot_xor_decision_boundary(xor_model, X_xor, y_xor)
```



Conclusion

The neural network learned the XOR gate with `100` percent accuracy and a very low loss of `0.0003`. Because XOR inputs cannot be separated by a single straight line, a basic model fails on this problem. Adding a hidden layer with `8` `tanh` units allowed the network to curve its decision boundary, cleanly separating the matching pairs (`0, 0` and `1, 1`) from the different pairs (`0, 1` and `1, 0`). After `500` epochs of training, the predicted probabilities came out almost exact at `0` and `1`, matching all four outputs of the truth table.